

Bias vs Variance (Overfitting)

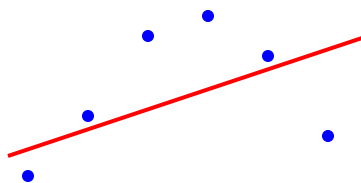
1. The Intuition: Goldilocks and the Three Models

Building a machine learning model is a balancing act.

- **Underfitting (High Bias):** The model is "too simple." It's like a student who didn't study and just guesses "C" on every question. It ignores the data.
- **Overfitting (High Variance):** The model is "too complex." It's like a student who memorized the exact answers to the practice test but fails the real exam because they didn't learn the concepts.
- **Just Right:** The model captures the underlying pattern but ignores the random noise.

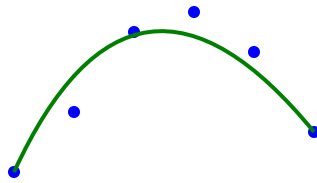
2. Visualization: The Tradeoff

Underfitting



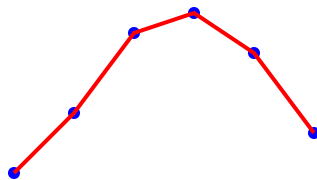
Model is too rigid (Linear).

Just Right



Captures the trend.

Overfitting



Memorizes noise.

3. The Math: Decomposition

We can mathematically break down the error of any model into three parts:

Bias-Variance Decomposition

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

- **Bias (Model Error):** Error from erroneous assumptions in the learning algorithm.
Source: The Model is too simple (Underfitting).
- **Variance (Model Sensitivity):** Error from sensitivity to small fluctuations in the training set.
Source: The Model is too complex (Overfitting).
- **Noise (Data Error):** The irreducible error inherent in the data itself.
Source: The Data quality/randomness (Cannot be fixed by the model).

Goal: Find the "Sweet Spot" where the sum of Bias and Variance is minimized.

4. Python Implementation: Polynomial Regression

We can force overfitting by using a high-degree polynomial (e.g., degree 9) on simple data.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt

def bias_variance_demo_polynomial_regression():
    """
    Illustrate underfitting/overfitting with polynomial regression.
    """
    # 1. Create Synthetic Data (Sine wave + Noise)
    rng = np.random.RandomState(0)
    X = np.linspace(0, 1, 100).reshape(-1, 1)
    y_true = np.sin(2 * np.pi * X).ravel()
    y = y_true + 0.3 * rng.randn(len(X)) # Add noise

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

    degrees = [1, 3, 15] # 1=High Bias, 3=Optimal, 15=High Variance

    plt.figure(figsize=(15, 5))

    for i, deg in enumerate(degrees):
        # Create pipeline
        model = Pipeline([
            ("poly", PolynomialFeatures(degree=deg)),
            ("lr", LinearRegression())
        ])
```

```

model.fit(X_train, y_train)

# Calculate Errors
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
train_err = mean_squared_error(y_train, train_pred)
test_err = mean_squared_error(y_test, test_pred)

# Visualization
ax = plt.subplot(1, 3, i + 1)
plt.setp(ax, xticks=(), yticks=())

# Plot Data
plt.scatter(X_train, y_train, edgecolor='b', s=20,
label="Samples")

# Plot Model Curve
X_plot = np.linspace(0, 1, 100).reshape(-1, 1)
plt.plot(X_plot, model.predict(X_plot), color='r',
label="Model")
plt.plot(X_plot, np.sin(2 * np.pi * X_plot), color='g',
alpha=0.5, linestyle='--', label="True Func")

plt.title(f"Degree {deg}\nTrain MSE: {train_err:.2f}, Test
MSE: {test_err:.2f}")
plt.legend(loc="best")

if deg == 1: plt.xlabel("Underfitting (High Bias)")
if deg == 3: plt.xlabel("Optimal")
if deg == 15: plt.xlabel("Overfitting (High Variance)")

plt.show()

bias_variance_demo_polynomial_regression()

```

5. Step-by-Step Math: Calculating Bias and Variance

Let's verify the decomposition equation

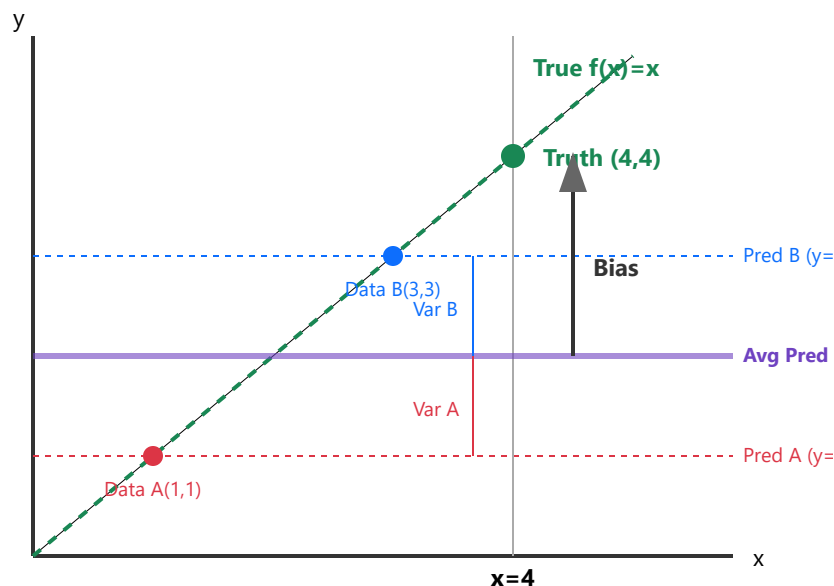
$$Error = Bias^2 + Variance$$

with a tiny manual example.

The Setup

- **True Function:** $f(x) = x$ (Identity line).
- **Test Point:** We want to predict for $x = 4$. The true value is $y = 4$.
- **The Algorithm:** A "Constant Model" that simply memorizes the training point.
 $\hat{f}(x) = y_{train}$.
- **The "Universe" of Data:** Suppose there are only 2 possible training datasets (equally likely):
 - **Dataset A:** Point (1, 1). Model becomes $\hat{f}_A(x) = 1$.
 - **Dataset B:** Point (3, 3). Model becomes $\hat{f}_B(x) = 3$.

Visualization of the Setup



Step 1: Calculate the "Average Model"

What does our model predict *on average* at $x = 4$?

- Prediction from Model A: 1

- Prediction from Model B: 3

$$\mathbb{E}[\hat{f}(4)] = \frac{1 + 3}{2} = 2$$

Step 2: Calculate Bias (Squared)

Bias is how far the *Average Model* is from the *Truth*.

$$Bias = \mathbb{E}[\hat{f}(4)] - f(4) = 2 - 4 = -2$$

$$\mathbf{Bias^2 = (-2)^2 = 4}$$

Step 3: Calculate Variance

Variance is how much the individual predictions vary from the *Average Prediction* (not the truth!).

- Model A deviation: $(1 - 2)^2 = (-1)^2 = 1$
- Model B deviation: $(3 - 2)^2 = (1)^2 = 1$

$$\mathbf{Variance = Average\ of\ deviations = \frac{1 + 1}{2} = 1}$$

Step 4: Verify Total Error (MSE)

Let's calculate the actual Mean Squared Error for each model and average it.

- Error for Model A: $(1 - 4)^2 = (-3)^2 = 9$
- Error for Model B: $(3 - 4)^2 = (-1)^2 = 1$

$$\mathbf{Total\ MSE = \frac{9 + 1}{2} = 5}$$

Does it add up?

$$Bias^2 + Variance = 4 + 1 = 5$$

Matches the Total MSE exactly!